

Estructuras de Datos

Comparadores

Búsqueda en Java

El método contains de la interface List

Retorna **true** si una lista contiene el element especificado

```
public class Thing {  
    public int value;  
    public Thing (int x) {  
        value = x;  
    }  
    equals (Thing x) {  
        if (x.value == value) {  
            return true;  
        }  
        return false;  
    }  
}
```

```
ArrayList<Thing> basket = new ArrayList<Thing>();  
Thing thing1 = new Thing(100);  
basket.add(thing1);  
Thing thing2 = new Thing(200);  
basket.add(thing2);  
basket.contains(thing2);
```

¿Qué se retorna?

El método contains de la interface List

Retorna **true** si una lista contiene el element especificado

```
public class Thing {  
    public int value;  
    public Thing (int x) {  
        value = x;  
    }  
}
```

```
ArrayList<Thing> basket = new ArrayList<Thing>();  
Thing thing1 = new Thing(100);  
basket.add(thing1);  
Thing thing2 = new Thing(200);  
basket.add(thing2);  
basket.contains(thing2);
```

¿Qué se retorna?

El método `contains` de la interface `List`

El uso del método `equals` hace que la comparación tenga que ser holística

La instancia, en su totalidad, es usualmente comparada

```
public Person lookupPerson (String personName) {  
    for (final Person person : persons) {  
        if (person.name.equals(personName)) {  
            // Found matching person  
            return person;  
        }  
    }  
    // Traversed whole list but did not find a matching person  
    return null;  
}
```

El método `contains` de la interface `List`

El uso del método `equals` hace que la comparación tenga que ser holística

La instancia, en su totalidad, es usualmente comparada



Estas dos personas son iguales solo si:

```
p1.nombres = p2.nombres;  
p1.apellidos = p2.apellidos;  
p1.edad = p2.edad;
```

Funciona solo cuando ya se conoce la instancia que se quiere encontrar

Sin embargo...

A veces podríamos querer buscar por atributos específicos

Ejemplo: Encontrar clientes de una categoría particular:

```
ArrayList<Customer> findByCategory (ArrayList<Customer> customers, String category) {  
  
    ArrayList<Customer> results = new ArrayList();  
    for (Customer customer : customers) {  
        if (customer.getCategory().equals(category) {  
            results.add(customer);  
        }  
    }  
    return results;  
}
```

Sin embargo...

A veces podríamos querer buscar por atributos específicos

Ejemplo: Encontrar clientes de una categoría particular:

```
ArrayList<Customer> findByCity (ArrayList<Customer> customers, String city) {  
  
    ArrayList<Customer> results = new ArrayList();  
    for (Customer customer : customers) {  
        if (customer.getCity().equals(city) {  
            results.add(customer);  
        }  
    }  
    return results;  
}
```


Sin embargo...

¿Cuál es el problema con esta estrategia?

¿Cómo podríamos generalizarla?

Usando comparadores

```
ArrayList<T> findAll (ArrayList<T> elements, Comparator cmp, T anotherElement) {  
    ArrayList<T> results = new ArrayList();  
    for (T element : elements) {  
        if (cmp.compare(element, anotherElement) == 0) {  
            results.add(element);  
        }  
    }  
    return results;  
}
```

La Interface Comparator<E>

Provee orden para colecciones de objetos que no tiene un orden *natural*

`int` compare (T object1, T object2)

Retorna un entero:

Valor	Significado
Negativo	<code>object1 < object2</code>
Cero	<code>object1 = object2</code>
Positivo	<code>object1 > object2</code>

Esto permite implementar la búsqueda en nuestro TDA:

`LinkedList<E> findAll (Comparator<E> cmp, E object)`

Formas de Utilizar Comparadores

1. Creando una instancia de la clase Comparator

```
Comparator<Customer> cmpByCity = new Comparator<Customer>() {  
    @Override  
    public int compare(Customer o1, Customer o2) {  
        if (o1.getCity().equals(o2.getCity())) {  
            return 0;  
        } else {  
            return 1;  
        }  
    }  
};
```

```
// usamos una instancia que contiene el criterio o valor que queremos buscar  
Customer customerGye = new Customer();  
customerGye.setCity("Guayaquil");
```

```
LinkedList<Customer> results = allCustomers.findAll(cmpByCity, customerGye);
```

2. Usando una clase anónima:

```
Customer customerGye = new Customer();  
customerGye.setCity("Guayaquil");
```

```
LinkedList<Customer> results = allCustomers.findAll(new Comparator<Customer>() {  
    @Override  
    public int compare(Customer o1, Customer o2) {  
        if (o1.getCity().equals(o2.getCity())) {  
            return 0;  
        } else {  
            return 1;  
        }  
    }  
}, customerGye);
```

Mediante una expresión lambda

```
Customer customerGye = new Customer();  
customerGye.setCity("Guayaquil");
```

```
LinkedList<Customer> results = allCustomers.findAll((Customer o1, Customer o2) -> {  
    if (o1.getCity().equals(o2.getCity())) {  
        return 0;  
    } else {  
        return 1;  
    }  
}, customerGye);
```